

Every money unit is a box, and I let the money flow as long as it does not flow out.

First thing to do is to read the text file and retrieve the map. I draw the map.

Secondly begin to pour money for every index of the map (inner indices are enough since the money in an outer index flow outside). Take the left upper corner of the map (the left upper index) I pour the money and fill the lakes until a money flow outside. Then I go next place to pour the money (next index). I add a unit of money, a box of money, to the index first. Then I let it flow until it stops flowing (as it is left nowhere else to go) or flows outside. It strictly moves if it has a place to move. It moves downwards first, if a index is a lower level, the money goes there. If there is no lower level place to go, it goes to a place where it did not go before. The places it went are marked in a matrix for every unit of money. Check if it went there before, if not and if it is the same level and if it is a neighbouring place where it can go, then go. There is priority to the neighboring of box's current index but the box can directly go to a neighboring index of an index that it went before. The unit of money can not do any other movement. Thus the loop ends.

This is similar to breadth first algorithms since the box flows to the neighboring indices only. All the indices are tried out and only then the box is added. Until the pond is filled. Then the box flows on top of the pond or finds another path. It finds a path until it flows outside or stays stable.

Thirdly, I calculate the score. As the lakes are filled, the added money boxes create piles of money on an empty map. The piles of money are lakes of money on map. Every pile is a lake and has a volume calculated by the sum of heights of the money piles in every index of the pile. As the piles are labeled, the label is marked on another empty map. Marked map is used for printing the final map with lake labels from A to ZZ. The labeling of the piles are similar to a breadth first algorithm too. It also labels only the neighbouring indices and runs the loop until no other left. Score is calculated by sum of sqrt of all pile volumes. Then printed in the end.

All the methods are defined in the second file. This file is for maps and methods. The methods retrieves data from the matrices that are initiated in the file.

Note: A lot of effort put into the code and algorithm. Some code are commented such that viewer can activate the code and see more output on console.

Assignment3FloodEverywhereAndAllCoordinates – Environment.java

signment3FloodEverywhereAndAllCoordinates > src > Environment > sumList

Main.java x Environment.java x input2.txt x trial.txt x input.txt x

drawE 9 results

```
110 //System.out.println("add: "+s);
111 String a=null;
112 String b=null;
113 int stoneCoordinateX=0;
114
115
116 if(s.length()<2){System.out.print("Not a valid step!");}
117
118 else if ((int)s.charAt(0)>96 && (int)s.charAt(0)<123 && (int)s.charAt(1)>96 && (int)s.charAt(1)<123) {
119     a=s.substring(0,2);
120     b=s.substring( beginIndex: 2);
121     for (int i = 0; i < coordinatesList.length; i++) {
122         if(coordinatesList[i].equals(a)){
123             //system.out.println("i: "+i);
124             stoneCoordinateX=i;
125         }
126     }
127     try {
128         int stoneCoordinateY = Integer.parseInt(b);
129         //System.out.println(b + " is an integer.");
130     } catch (Exception e) {
131         //System.out.println("Invalid integer");
132     }
133 }
```

Run: Main x

	a	b	c	d	e	f	g
0	3	3	3	3	3	3	3
1	3	A	A	3	A	A	3
2	3	3	3	A	3	3	3
3	3	3	3	4	4	4	4
4	3	B	B	4	C	C	4
5	3	3	3	4	4	4	4
6	3	3	3	3	3	3	2
	a	b	c	d	e	f	g

Final score: 6.80

Process finished with exit code 0